

Plot in a Pot

Um Editor Visual e Simulador para Histórias
Interativas e Lógica Narrativa

Miguel Gomes Silva

Student No. 2221462

Tomás Alexandre dos Santos Reis Lopes

Student No. 2222970

Supervisores: Anabela Gonçalves Rodrigues Marto

*Professora Adjunta, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Miguel Cerdeira Marreiros Negrão

*Professor Adjunto, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Nelson Martins Ferreira

*Professor Adjunto, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Escola Superior de Tecnologia e Gestão

Departamento de Engenharia Informática

Licenciatura em Engenharia Informática

UC de Projeto Informático

Trabalho de Projeto da unidade curricular de Projeto Informático realizado sob a orientação da Professora Doutora Anabela Gonçalves Rodrigues Marto e do Professor Doutor Miguel Cerdeira Marreiros Negrão e do Professor Doutor Nelson Martins Ferreira

Leiria, Junho 2026

Plot in a Pot

Copyright © 2026 - Miguel Gomes Silva, Escola Superior de Tecnologia e Gestão.

A presente dissertação é um trabalho original, elaborado exclusivamente para este fim, tendo sido devidamente citados todos os autores cujos estudos contribuíram para a sua elaboração. É permitida a sua reprodução parcial com indicação do autor e referência ao grau, ano letivo, instituição—*Politécnico de Leiria*—e data da defesa pública.

O presente trabalho beneficiou da utilização do modelo *IPLeiria-Thesis*.

Agradecimentos

Gostaríamos de expressar o nosso mais sincero agradecimento a todos aqueles que, direta ou indiretamente, contribuíram para a realização deste projeto informático e para a elaboração deste relatório.

À nossa orientadora, Professora Doutora Anabela Gonçalves Rodrigues Marto, pelo voto de confiança, paciência, constante incentivo e precioso apoio técnico e científico manifestados ao longo de todas as fases deste trabalho. A sua orientação foi de extrema importância para a superação dos desafios metodológicos e organizacionais.

Aos nossos coorientadores, Professor Doutor Miguel Cerdeira Marreiros Negrão e Professor Doutor Nelson Martins Ferreira, pelas valiosas discussões, sugestões práticas e comentários de revisão rigorosos que ajudaram a elevar a qualidade técnica e arquitetural da nossa solução.

À Escola Superior de Tecnologia e Gestão (ESTG) do Politécnico de Leiria, por disponibilizar os recursos académicos e o ambiente propício ao nosso desenvolvimento pessoal e profissional ao longo da Licenciatura em Engenharia Informática.

Aos participantes que voluntariamente colaboraram na bateria de testes de usabilidade e acessibilidade, cujo feedback construtivo e sincero foi inestimável para a validação e refinamento prático do software.

Por fim, expressamos um agradecimento muito especial às nossas famílias e amigos, pelo apoio incondicional, incentivo constante e paciência demonstrados durante este percurso académico.

Resumo

O desenvolvimento de ficção interativa e jogos baseados em escolhas enfrenta sérios desafios de usabilidade e integridade estrutural à medida que a ramificação narrativa cresce. Embora existam ferramentas populares de autoria como o Twine, a maioria carece de suporte nativo integrado para gestão de localização multi-idioma e de motores de validação estática de caminhos lógicos para detetar inconsistências (como becos sem saída e cenas órfãs) antes da fase de compilação ou execução. O presente trabalho de projeto propõe o *Plot in a Pot* (PiaP), um editor visual e simulador baseado em grafos que unifica a autoria de narrativas escritas no formato aberto Tweep 3 (SugarCube) com um motor integrado de tradução de textos e validação estática de caminhos.

Implementado em React e no ecossistema React Flow, o PiaP destaca-se pela sua estética neo-brutalista de alto contraste visual, concebida para mitigar barreiras de acessibilidade cognitiva e cromática para autores com daltonismo severo (como deuteranopia e protanopia). O sistema integra: (i) um parser síncrono bidirecional para ler e reconstruir topologias Tweep 3 em tempo real; (ii) uma matriz de localização baseada em grelha bidimensional com importação e exportação de CSV; (iii) um motor de validação estática suportado por algoritmos de Pesquisa em Largura (BFS) para analisar a acessibilidade de cenas; (iv) um modo de simulação interativo dotado de um agente inteligente autónomo (*Smart Walker*) que utiliza procura por novidade (*Novelty Search*) para percorrer e testar caminhos automaticamente; e (v) arestas com rota curva parametrizada por splines cúbicas de Catmull-Rom.

A avaliação qualitativa e quantitativa do sistema foi efetuada através de testes com seis utilizadores com diferentes perfis técnicos e de acessibilidade. Os resultados revelaram uma taxa média de conclusão de tarefas de 91,7% e uma elevada usabilidade e satisfação, validando a eficácia do *Plot in a Pot* como uma ferramenta viável e robusta para autores e *game designers* na prototipagem e desenvolvimento de narrativas interativas complexas.

Palavras-Chave: Ficção Interativa, Editores de Grafos, Localização de Jogos, Acessibilidade Visual, Validação Estática de Fluxo, Twine, SugarCube.

Abstract

The development of interactive fiction and choice-based games faces serious challenges in usability and structural integrity as the narrative branching expands. Although popular authoring tools like Twine exist, most lack integrated native support for multi-language localization management and static path-validation engines to detect inconsistencies (such as dead ends and orphan scenes) before compilation or execution. This project work proposes *Plot in a Pot* (PiaP), a graph-based visual editor and simulator that unifies the authoring of interactive stories written in the open Tweep 3 format (SugarCube) with an integrated translation matrix and static path-validation engine.

Implemented in React and using the React Flow ecosystem, PiaP stands out for its high-contrast neo-brutalist visual style, specifically designed to mitigate cognitive and chromatic accessibility barriers for authors with severe color blindness (such as deuteranopia and protanopia). The system features: (i) a synchronous bidirectional parser to read and reconstruct Tweep 3 topologies in real time; (ii) a translation matrix layout with CSV import and export capabilities; (iii) a static validation engine powered by Breadth-First Search (BFS) algorithms to audit scene reachability; (iv) an interactive simulation mode equipped with an autonomous agent (*Smart Walker*) that leverages novelty search heuristics to automatically test paths; and (v) customizable edge routing parameterized by Catmull-Rom cubic splines.

Qualitative and quantitative evaluation was conducted through user testing with six participants of varying technical backgrounds and accessibility needs. The results showed a 91.7% average task completion rate alongside high levels of usability and satisfaction, validating the effectiveness of *Plot in a Pot* as a viable and robust tool for authors and game designers during the prototyping and development of complex interactive narratives.

Keywords: Interactive Fiction, Graph Editors, Game Localization, Visual Accessibility, Static Flow Validation, Twine, SugarCube.

Declaração sobre o Uso de Inteligência Artificial

i Orientação Para a Escrita

É uma boa prática académica indicar brevemente como, porquê e quando a IA foi utilizada. Explicar as ações tomadas e as razões que lhes estão subjacentes promove a reflexão crítica, aprofunda a compreensão do papel da IA na aprendizagem e ajuda a avaliar o seu impacto na qualidade e integridade do trabalho. Esta página pode ser removida definindo a opção `aiack` como `false` nas opções da classe do documento.

i Exemplo de Utilização

Reconheço a utilização do Gemini (<https://gemini.google.com>) para aperfeiçoar o tom académico e melhorar a precisão linguística deste trabalho, incluindo aspectos de gramática, pontuação e vocabulário.

Descrição da Utilização

Instrução 1: Num parágrafo, descreva simplesmente o que é o LaTeX.

Resposta 1: O LaTeX é um sistema de composição de alta qualidade, normalmente utilizado para produzir documentos científicos e matemáticos, devido ao seu poderoso manuseamento de fórmulas e bibliografias. Ao contrário dos processadores de texto, utiliza ficheiros de texto simples com etiquetas de marcação para definir a estrutura e a formatação de um documento, permitindo um controlo preciso da disposição e do aspeto. É especialmente popular no meio académico e na investigação para a criação de teses, artigos de revistas e apresentações.

Conteúdo

<i>Lista de Figuras</i>	xii
<i>Lista de Tabelas</i>	xiv
<i>Siglas</i>	xvi

1	Desenvolvimento e Avaliação	1
1.1	Cronograma e Fases de Desenvolvimento	1
1.2	Estrutura do Código e Detalhes de Implementação	2
1.2.1	Componentes de Interface (src/components/)	3
1.2.2	Módulos de Lógica e Utilitários (src/utils/)	10
1.2.3	Fluxo de Dados e Reatividade	12
1.3	Testes de Conversão e Comparação de Modelos de Linguagem (IA Local)	13
1.4	Análise de Desempenho e Complexidade Algorítmica	14
1.4.1	Complexidade Algorítmica Teórica	14
1.4.2	Benchmarks de Desempenho Empíricos	15
1.5	Metodologia dos Testes de Utilizador	15
1.5.1	Perfil dos Participantes	16
1.5.2	Tarefas Avaliadas	16
1.6	Resultados dos Testes e Desenvolvimento Iterativo	16
1.6.1	Primeira Leva: Protótipo Inicial (Abril de 2026)	17
1.6.2	Segunda Leva: Protótipo Intermédio (Maio de 2026)	17
1.6.3	Terceira Leva: Protótipo Final (Junho de 2026)	17
1.6.4	Feedback Qualitativo e Usabilidade Visual	19

Apêndices

A	Manual de Utilização e Instalação	24
A.1	Requisitos do Sistema e Instalação Local	24
A.1.1	Instalação do Editor Web	24
A.1.2	Configuração do Servidor de IA Local (Ollama)	25
A.2	Manual de Utilização do Editor Gráfico	25
A.2.1	O Canvas Interativo	25
A.2.2	Criação e Edição de Nós Narrativos (Cenas)	26
A.2.3	Criação de Ligações e Arestas de Escolha	26

A.2.4	Agrupamento em Zonas (Capítulos)	26
A.3	Programação de Lógica Narrativa e Variáveis	27
A.3.1	Criação de Variáveis (Figma-Style Tokens)	27
A.3.2	Edição de Lógica condicional por Código	27
A.3.3	Editor de Blocos Visuais	28
A.4	Tradução e Localização Multi-idioma	28
A.4.1	Configurar Idiomas	28
A.4.2	Editar Textos na Matriz	28
A.4.3	Importação e Exportação de Ficheiros CSV	28
A.5	Validação Lógica e Simulação de Jogo	29
A.5.1	O Painel de Erros de Validação Estática	29
A.5.2	Simulação Ativa (PlayMode) e Testes com o Smart Walker	29
A.6	Operações de Ficheiros e Publicação do Projeto	29
A.6.1	Importar Ficheiros Twee 3	29
A.6.2	Exportar e Compilar a História	30
B	Guia de Sintaxe e Modelagem de Macros Lógicas	31
B.1	Estrutura de um Documento Twee 3	31
B.1.1	Passagens Especiais de Sistema	31
B.1.2	Passagens Narrativas (Cenas)	32
B.2	Sintaxe de Macros Lógicas (SugarCube)	32
B.2.1	Macro de Atribuição: <<set>>	32
B.2.2	Macro de Controlo Condicional: <<if>>	32
B.3	Padrões Comuns de Modelação Narrativa	33
B.3.1	Padrão 1: Inventário e Portas Trancadas	33
B.3.2	Padrão 2: Contador de Tentativas e Fim de Jogo	33
B.3.3	Padrão 3: Caminhos Ocultos e Nós Secretos	34
 Anexos		
L	Especificação Técnica Normativa do Formato Twee 3	38
L.1	Sintaxe Geral e Passagens	38
L.1.1	Sintaxe do Cabeçalho da Passagem	38
L.2	Especificação de Metadados de Posicionamento	39
L.2.1	Campos de Geometria JSON	39
L.3	Especificação de Hiperligações e Transições	39

Lista de Figuras

1.1	Interface gráfica do canvas de grafos do Plot in a Pot, demonstrando a estética neo-brutalista de alto contraste com nós de cena e agrupamentos visuais (Zonas).	6
1.2	Ecrã da sandbox de simulação (PlayMode), ilustrando a execução em tempo real da história com o painel de depuração de variáveis lógicas.	7
1.3	Matriz de localização integrada no editor, exibindo a grelha de chaves de texto e a respetiva tradução direta para os idiomas selecionados.	9

Lista de Tabelas

1.1	Cronograma detalhado das fases de desenvolvimento do projeto.	1
1.2	Complexidade algorítmica teórica dos subsistemas nucleares.	14
1.3	Resultados empíricos de benchmarks de desempenho (em milissegundos). . .	15
1.4	Taxa de sucesso e tempo de conclusão (em minutos) das tarefas por utilizador (Leva Final).	18
1.5	Pontuação obtida no questionário System Usability Scale (SUS) na leva final.	19

Siglas

BFS	Pesquisa em Largura (Breadth-First Search). <i>(p. 34)</i>
IFTF	Interactive Fiction Technology Foundation. <i>(p. 38, 39)</i>
JSON	JavaScript Object Notation. <i>(p. 31)</i>
PiaP	Plot in a Pot. <i>(p. 24, 25, 27, 28, 30, 31, 32, 33, 34, 38, 39)</i>

1

Desenvolvimento e Avaliação

Este capítulo detalha o processo de desenvolvimento prático do *Plot in a Pot*, descrevendo as fases de implementação, o cronograma cronológico do projeto e a metodologia e resultados dos testes de utilizador realizados para validar a usabilidade e acessibilidade do sistema.

1.1 Cronograma e Fases de Desenvolvimento

O projeto foi executado ao longo de um período aproximado de três meses, utilizando uma abordagem de desenvolvimento incremental assente em prototipagem rápida. O cronograma de implementação dividiu-se em cinco fases fundamentais, estruturadas conforme se descreve de seguida:

Tabela 1.1: *Cronograma detalhado das fases de desenvolvimento do projeto.*

Fase	Descrição das Atividades Realizadas	Intervalo de Datas
Fase 1	Levantamento de requisitos e desenho conceitual da arquitetura.	17/03/2026 – 31/03/2026
Fase 2	Programação do núcleo (<i>core</i>) lógico: parser Twee 3 e validador BFS.	01/04/2026 – 25/04/2026
Fase 3	Interface e visualização: integração do React Flow e <i>waypoints</i> nas arestas.	26/04/2026 – 20/05/2026
Fase 4	Integração de IA local (Ollama) e otimização de acessibilidade visual.	21/05/2026 – 10/06/2026
Fase 5	Testes com utilizadores, correção de erros e refinamento final.	11/06/2026 – 25/06/2026

Fase 1 – Desenho e Requisitos: Definição das especificações funcionais e não funcionais do sistema. Elaboração do diagrama de arquitetura modular para separar a camada gráfica do processamento de grafos.

Fase 2 – Motor Lógico: Implementação da biblioteca `tweeParser.js` para ler e estruturar o formato Twee 3 em memória como uma lista de adjacências bidirecional,

e do motor de validação estática assente em travessias BFS.

Fase 3 – Componentes de Canvas: Acoplamento do React Flow e desenho de nós customizados, incluindo a lógica de grupos (Zonas) com gestão de pointer events, e as arestas editáveis baseadas em curvas Spline Catmull-Rom.

Fase 4 – IA e Acessibilidade: Configuração de chamadas HTTP para o Ollama local e validação da conversão de texto livre em nós Tweep. Ajustes no design de Tailwind CSS para implementar a estética neo-brutalista de alto contraste e a redundância de traços para daltónicos.

Fase 5 – Validação Prática: Realização de testes estruturados com utilizadores finais para identificar bloqueios cognitivos na interface e falhas de usabilidade.

1.2 Estrutura do Código e Detalhes de Implementação

Para além da fundamentação teórica e da arquitetura abstrata do sistema, o processo de desenvolvimento envolveu a estruturação de uma base de código modular e reativa baseada em padrões modernos do ecossistema React 19. O software está organizado física e logicamente em torno de uma diretoria centralizada (`/src/`), estruturada de acordo com o seguinte esquema de diretórios:

```
src/
+- components/
| +- AiImportModal.jsx      # Importação de histórias com IA
| +- ConnectionModal.jsx    # Ligação ao Ollama local
| +- DataPanel.jsx          # Painel lateral de dados brutos
| +- DeleteConfirmModal.jsx # Confirmação de remoção de nós
| +- EditTranslationModal.jsx # Edição rápida de traduções
| +- EditableEdge.jsx       # Arestas com splines e waypoints
| +- ExportModal.jsx        # Exportação de ficheiros
| +- Inspector.jsx         # Painel lateral de edição de nós
| +- Minimap.jsx           # Navegação rápida pelo canvas
| +- PlayMode.jsx          # Sandbox do simulador de jogo
| +- PlaythroughTutorial.jsx # Tutorial interativo sequencial
| +- Popout.jsx            # Janela flutuante de visualização
| +- SettingsModal.jsx     # Configurações globais do editor
| +- StoryNode.jsx         # Componente visual do nó de cena
| +- TranslationMatrix.jsx  # Grelha de tradução multi-idioma
| +- ValidationModal.jsx    # Diálogo de erros lógicos (BFS)
| +- VariablesModal.jsx     # Gestão de variáveis (estilo Figma)
| +- VisualBlocksEditor.jsx # Programação visual por blocos
| \- ZoneNode.jsx          # Retângulo de agrupamento visual
+- contexts/               # Contextos de estado global React
+- utils/
```

```

| +- aiGenerator.js          # API REST para Ollama/Gemini
| +- graphMath.js           # Controlo geométrico e Splines
| +- logicParser.js         # Parser AST de macros SugarCube
| +- storySimulator.js      # Máquina de estados da história
| +- storyTraversal.js      # Exploração heurística (Smart Walker)
| +- storyValidator.js      # Validação estática com BFS
| +- sugarcubeLogic.js      # Compilador JIT de macros Twine
| \- tweeParser.js         # Parser bidirecional Tweep 3
+- App.jsx                  # Controlador de estado global
\- index.js                 # Entrada da aplicação

```

1.2.1 Componentes de Interface (src/components/)

Nesta secção, detalham-se os aspetos de desenvolvimento e engenharia aplicados à implementação dos principais componentes visuais da aplicação.

App.jsx: O Controlador Reativo Central e Sincronização de Estado

O ficheiro `App.jsx` atua como o orquestrador principal da aplicação. Ele gerencia o estado global da narrativa (vetores de nós e arestas do React Flow), as variáveis de estado definidas pelo utilizador e a sincronização com os diferentes modais. Para além de implementar as rotinas de ligação com o React Flow (`onNodesChange`, `onEdgesChange`, `onConnect`), gere a pilha de histórico (`undoStack` e `redoStack`) para permitir o retrocesso ou avanço atómico de edições na interface gráfica. Dado que o React Flow atualiza objetos de nós e arestas por referência, a criação de instantâneos (*snapshots*) do estado requer uma clonagem profunda (*deep clone*) para evitar que estados futuros partilhem a mesma referência de memória com o histórico, como demonstrado na Listagem 1.

StoryNode.jsx e ZoneNode.jsx: Nós Customizados no React Flow

Os nós gráficos que habitam o canvas estendem os componentes base do React Flow. O `StoryNode.jsx` é encarregue de renderizar o título da passagem, o extrato de texto da história e as portas de ligação (*handles*) de entrada e saída. O `ZoneNode.jsx` implementa o agrupamento visual de capítulos. Do ponto de vista de desenvolvimento CSS, a implementação das Zonas colocou um desafio técnico complexo: quando nós de cena são arrastados para dentro do retângulo da Zona, o `ZoneNode` não deve interceptar eventos do rato que pertencem ao canvas de fundo ou aos nós filhos. Para resolver este conflito de sobreposição espacial, aplicou-se a propriedade `pointer-events: none` ao contêiner da Zona e ativou-se `pointer-events: all` exclusivamente para as bordas de redimensionamento e barra de título, conforme ilustrado na Listagem 2.

```

1 // Registo de snapshots e funções de controlo em App.jsx
2 const takeSnapshot = useCallback(() => {
3   setPast(prev => [
4     ...prev.slice(-50),
5     {
6       nodes: JSON.parse(JSON.stringify(nodesRef.current)),
7       edges: JSON.parse(JSON.stringify(edgesRef.current))
8     }
9   ]);
10  setFuture([]);
11 }, []);
12
13 const undo = useCallback(() => {
14   if (past.length === 0) return;
15   const previous = past[past.length - 1];
16   setFuture(prev => [
17     {
18       nodes: JSON.parse(JSON.stringify(nodesRef.current)),
19       edges: JSON.parse(JSON.stringify(edgesRef.current))
20     },
21     ...prev
22   ]);
23   setPast(prev => prev.slice(0, -1));
24   setNodes(previous.nodes);
25   setEdges(previous.edges);
26 }, [past]);

```

Listagem 1: Mecanismo de clonagem profunda e gestão da pilha de histórico em App.jsx.

```

1 <!-- Estrutura simplificada do componente ZoneNode -->
2 <div
3   className="zone-node-container"
4   style={{ pointerEvents: 'none' }}
5 >
6   <div
7     className="zone-node-header"
8     style={{ pointerEvents: 'all' }}
9   >
10    <h3>{data.title}</h3>
11  </div>
12  <div
13    className="zone-node-body"
14    style={{ border: '2px dashed #000' }}
15  >
16    /* Os nós filhos são renderizados no topo e respondem a cliques */
17  </div>
18 </div>

```

Listagem 2: Resolução de eventos de rato através de regras CSS em ZoneNode.jsx.

EditableEdge.jsx: Arestas com Splines de Catmull-Rom e Waypoints

Para evitar arestas retilíneas que cruzam e cobrem os nós narrativos, desenvolveu-se o componente `EditableEdge.jsx`. O cálculo do traçado da aresta baseia-se na interpolação Spline de Catmull-Rom. O utilizador pode efetuar um duplo clique sobre a linha para introduzir um ponto de controlo (*waypoint*) geométrico. A Listagem 3 detalha a transformação das coordenadas dos waypoints num caminho em formato SVG.

```

1 // Geração da curva SVG com interpolação de pontos geométricos
2 export function getCatmullRomPath(points) {
3   if (points.length < 2) return '';
4   let path = `M ${points[0].x} ${points[0].y}`;
5   for (let i = 0; i < points.length - 1; i++) {
6     const p0 = points[i - 1] || points[i];
7     const p1 = points[i];
8     const p2 = points[i + 1];
9     const p3 = points[i + 2] || p2;
10
11     // Cálculo dos pontos de controlo Bézier baseados em Catmull-Rom
12     const cp1x = p1.x + (p2.x - p0.x) / 6;
13     const cp1y = p1.y + (p2.y - p0.y) / 6;
14     const cp2x = p2.x - (p3.x - p1.x) / 6;
15     const cp2y = p2.y - (p3.y - p1.y) / 6;
16
17     path += ` C ${cp1x} ${cp1y}, ${cp2x} ${cp2y}, ${p2.x} ${p2.y}`;
18   }
19   return path;
20 }
```

Listagem 3: Algoritmo de interpolação e desenho de curvas no componente *EditableEdge.jsx*.

Inspector.jsx: Painel de Propriedades e Mapeamento Reverso

O painel lateral `Inspector.jsx` é montado dinamicamente em resposta à seleção de nós no canvas. Ele lê e sincroniza os metadados do nó selecionado. Para facilitar a navegação do autor em grafos complexos, o componente analisa a lista de adjacências bidirecional em tempo de execução para calcular os caminhos de entrada e de saída que se conectam à cena ativa. A Listagem 4 demonstra como este mapeamento reverso é computado.

PlayMode.jsx: Sandbox de Simulação com Isolamento Dinâmico

O ecrã de simulação `PlayMode.jsx` disponibiliza uma área de testes livre de interferências. Quando inicializado, os blocos de código CSS definidos pelo autor nas cenas com a tag correspondente são compilados e inseridos no cabeçalho do DOM sob a forma de uma tag `<style>`. De igual modo, a execução dos scripts de lógica definidos em JavaScript é isolada das variáveis de execução interna da interface através da criação de funções anónimas com escopo restrito (Listagem 5).

```

1 // Cálculo de caminhos incidentes no painel lateral
2 const getConnections = (nodeId, edges, nodes) => {
3   const incoming = edges
4     .filter(edge => edge.target === nodeId)
5     .map(edge => {
6       const srcNode = nodes.find(n => n.id === edge.source);
7       return { id: edge.source, label: srcNode?.data?.label || edge.source };
8     });
9
10  const outgoing = edges
11    .filter(edge => edge.source === nodeId)
12    .map(edge => {
13      const tgtNode = nodes.find(n => n.id === edge.target);
14      return { id: edge.target, label: tgtNode?.data?.label || edge.target };
15    });
16
17  return { incoming, outgoing };
18 };

```

Listagem 4: Mapeamento dinâmico de nós predecessores e sucessores em *Inspector.jsx*.

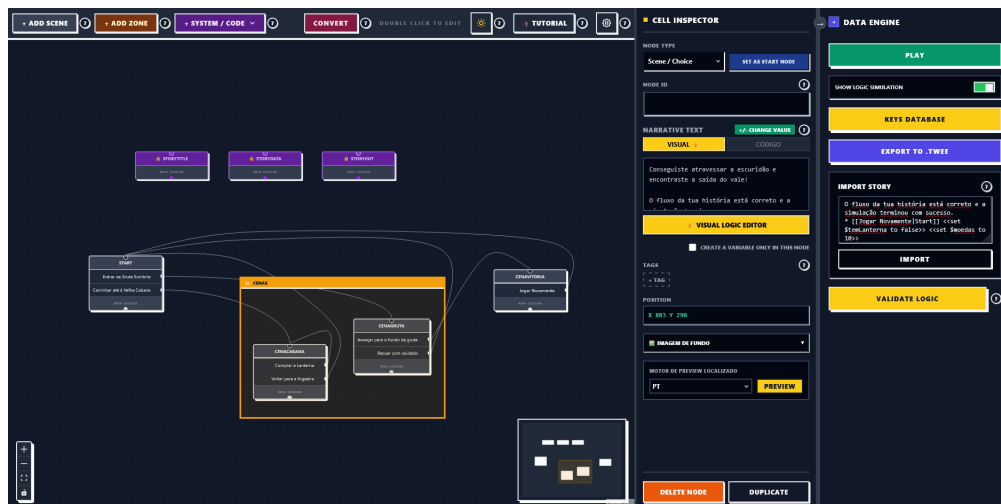


Figura 1.1: Interface gráfica do canvas de grafos do Plot in a Pot, demonstrando a estética neo-brutalista de alto contraste com nós de cena e agrupamentos visuais (Zonas).

```
1 // Execução controlada de scripts do utilizador
2 const executeScriptInSandbox = (scriptContent, gameState) => {
3   try {
4     const sandbox = new Function('state', `
5       with(state) {
6         ${scriptContent}
7       }
8     `);
9     // Passa o clone profundo do estado do jogo para isolamento
10    const stateClone = JSON.parse(JSON.stringify(gameState));
11    sandbox(stateClone);
12    return stateClone;
13  } catch (error) {
14    console.error("Erro na execução do script do utilizador:", error);
15    return gameState;
16  }
17 };
```

Listagem 5: Mecanismo de execução de scripts de lógica com isolamento de contexto no *PlayMode.jsx*.

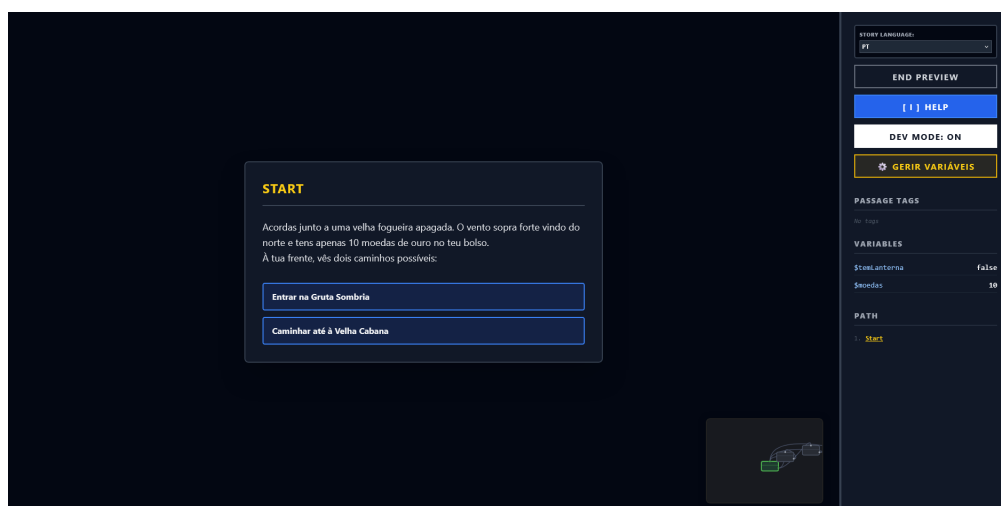


Figura 1.2: Ecrã da sandbox de simulação (*PlayMode*), ilustrando a execução em tempo real da história com o painel de depuração de variáveis lógicas.

VariablesModal.jsx: Refatorização por Expressões Regulares

A gestão de variáveis globais é centralizada no `VariablesModal.jsx`. Caso um autor mude o nome de uma variável (ex: de `$chave` para `$chave_dourada`), o sistema precisa de atualizar de forma atômica todas as ocorrências textuais presentes nas condições lógicas de todos os nós para evitar quebra de referências. Para tal, o componente varre as propriedades da história aplicando uma expressão regular com barreira de palavras (`\b`) para evitar substituições parciais de termos (Listagem 6).

```

1 // Substituição em lote de variáveis nos textos dos nós
2 const refactorVariableInNodes = (oldName, newName, currentNodes) => {
3   // Regex que isola a variável escapando o símbolo $
4   const escapedOld = oldName.replace(/[-\/\\^$*+?.()|[\]{}]/g, '\\$&');
5   const regex = new RegExp(escapedOld + '\\b', 'g');
6
7   return currentNodes.map(node => {
8     const text = node.data.content || '';
9     if (regex.test(text)) {
10      return {
11        ...node,
12        data: { ...node.data, content: text.replace(regex, newName) }
13      };
14    }
15    return node;
16  });
17 };

```

Listagem 6: Refatorização automática de variáveis lógicas baseada em Regex no `VariablesModal.jsx`.

TranslationMatrix.jsx: Tradução Tabular e Importação/Exportação CSV

A tabela de localização `TranslationMatrix.jsx` expõe a grelha de tradução de strings da história. Cada linha representa uma chave (o ID de uma cena narrativa) e cada coluna representa um idioma (ex: `pt`, `en`, `es`). A alteração de uma célula propaga-se dinamicamente para o dicionário do `i18next` na memória local. O componente disponibiliza suporte a exportação e importação de ficheiros de localização em formato CSV em conformidade com as regras de escape de aspas da norma RFC 4180 (Listagem 7).

PlaythroughTutorial.jsx e VisualBlocksEditor.jsx: Aprendizagem e Programação por Blocos

O `PlaythroughTutorial.jsx` fornece uma experiência guiada para utilizadores novatos, intercetando as interações com o canvas e forçando o cumprimento de etapas lógicas. Por sua vez, o `VisualBlocksEditor.jsx` é um editor visual por arrasto de blocos que traduz logicamente comandos aritméticos e de atribuição em texto corrido compatível com `SugarCube`, conforme demonstrado no parser recursivo da Listagem 8.

```

1 // Conversão de texto CSV bruto numa matriz de tradução
2 const parseCSVToTranslations = (csvText) => {
3   const rows = [];
4   let currentRow = [];
5   let currentCell = '';
6   let inQuotes = false;
7
8   for (let i = 0; i < csvText.length; i++) {
9     const char = csvText[i];
10    const nextChar = csvText[i + 1];
11
12    if (char === '"') {
13      if (inQuotes && nextChar === '"') {
14        currentCell += '"'; // Aspas escapadas
15        i++;
16      } else {
17        inQuotes = !inQuotes; // Alternar estado de aspas
18      }
19    } else if (char === ',' && !inQuotes) {
20      currentRow.push(currentCell.trim());
21      currentCell = '';
22    } else if ((char === '\n' || char === '\r') && !inQuotes) {
23      if (char === '\r' && nextChar === '\n') i++;
24      currentRow.push(currentCell.trim());
25      rows.push(currentRow);
26      currentRow = [];
27      currentCell = '';
28    } else {
29      currentCell += char;
30    }
31  }
32  return rows;
33 };

```

Listagem 7: Parser de importação de ficheiros CSV com suporte à norma RFC 4180 em *TranslationMatrix.jsx*.

KEY	PT (source)	EN	ACT
story_intro	Acorde à entrada de um vale gelado. Sentes os kolos pesados. À tua frente, um portão de ferro.	You wake up at the entrance of a cold valley; your pockets feel heavy. In front of you, a gate.	✖
story_market	Um mercador sombrio sorri para ti por trás do seu balcão de madeira. "Procuras uma entrada para si."	A shady merchant smiles at you from behind his wooden counter. "Looking for a way into the..."	✖
story_gate	O portão de ferro é gelado ao toque. Mãos mágicas pesadas impedem qualquer um de o forçar.	The iron gate is ice-cold to the touch. Heavy magical runes prevent anyone from forcing it open.	✖
story_inside	O portão abre-se com um rangido! Passas para o interior de gruta de cristal labiríntico.	The gate creaks open! You step into the glowing crystal cavern. You successfully bypassed the...	✖
choices_return_gate	Voltar para o portão de ferro	Head back to the iron gate	✖
choices_return_market	Voltar ao mercador	Go back to the merchant	✖
choices_key_open	Inserir a Chave da Memória na fechadura	Insert the Dragon Key into the lock	✖

Figura 1.3: Matriz de localização integrada no editor, exibindo a grelha de chaves de texto e a respetiva tradução direta para os idiomas selecionados.

```

1 // Tradução recursiva de blocos visuais para macros textuais
2 const compileBlockToText = (block) => {
3   if (!block) return '';
4   switch (block.type) {
5     case 'set_variable':
6       return `<<set ${block.variable} = ${compileBlockToText(block.value)}>>`;
7     case 'condition_if':
8       return `<<if
9         ${compileBlockToText(block.condition)}>>${block.thenText}<<endif>>`;
10    case 'logical_operator':
11      return `(${compileBlockToText(block.left)} ${block.op}
12        ${compileBlockToText(block.right)})`;
13    case 'literal_number':
14      return String(block.val);
15    default:
16      return '';
17  }
18 };

```

Listagem 8: *Conversão de blocos lógicos estruturados em macros textuais SugarCube em VisualBlocksEditor.jsx.*

1.2.2 Módulos de Lógica e Utilitários (src/utils/)

Os utilitários contêm os algoritmos semânticos, os parsers e a inteligência estrutural da plataforma, operando fora do ciclo direto de renderização do DOM:

tweeParser.js: Parser Bidirecional Twee 3

O ficheiro `tweeParser.js` é responsável por converter o layout visual do React Flow numa representação textual estável em formato Twee 3 e vice-versa. Durante a leitura, o parser interpreta os metadados JSON associados a cada passagem (que incluem a posição absoluta no ecrã e as tags) e processa as Zonas. Caso um nó seja filho de uma Zona de Agrupamento, a sua coordenada é traduzida de absoluta no ecrã para relativa ao canto superior esquerdo da zona mãe (Listagem 9).

storyValidator.js e storyTraversal.js: Travessia BFS e Exploração de Estados

A verificação estática do grafo de estados utiliza travessias BFS especiais em `storyTraversal.js`. Para evitar a explosão de estados e ciclos infinitos causados por incrementos repetitivos de variáveis lógicas, o ciclo principal do algoritmo limita as visitas redundantes ao mesmo nó de cena a um máximo de $K = 10$ estados lógicos distintos. Se este limiar for atingido, o ramo é podado e emite-se um alerta estrutural (Listagem 10).

aiGenerator.js: Assistente de IA Local e Prompting

A ponte de comunicação com os servidores locais (Ollama na porta 11434) e APIs proprietárias externas é mantida pelo ficheiro `aiGenerator.js`. O módulo envia pedidos assíncronos REST empacotando os textos literários livres fornecidos pelo utilizador e

```

1 // Conversão de coordenadas na leitura de passagens Twee 3
2 export function parsePassageMetadata(metaString, zoneMap) {
3   try {
4     const meta = JSON.parse(metaString || '{}');
5     let x = meta.position?.x ?? 0;
6     let y = meta.position?.y ?? 0;
7     const parentId = meta.parentId;
8
9     // Se pertencer a uma Zona, converte a posição para coordenada relativa
10    if (parentId && zoneMap.has(parentId)) {
11      const zone = zoneMap.get(parentId);
12      x = x - (zone.position?.x ?? 0);
13      y = y - (zone.position?.y ?? 0);
14    }
15    return { x, y, parentId, width: meta.size?.width, height: meta.size?.height
16      };
17  } catch (e) {
18    return { x: 0, y: 0 };
19  }

```

Listagem 9: *Processamento de coordenadas absolutas e relativas de nós e Zonas em tweeParser.js.*

```

1 // Travessia e controlo de ciclos por limite de estados visitados
2 while (queue.length > 0) {
3   const { nodeId, state, path } = queue.shift();
4   const currentNode = nodeMap.get(nodeId);
5   if (!currentNode) continue;
6
7   const newState = applyModifiers(currentNode.data.content, state);
8   const stateStr = JSON.stringify(newState);
9
10  if (!visitedStates.has(nodeId)) visitedStates.set(nodeId, []);
11  if (visitedStates.get(nodeId).includes(stateStr)) continue;
12
13  // Imposição do limite regulador de segurança K = 10
14  if (visitedStates.get(nodeId).length >= 10) {
15    console.warn(`Ciclo infinito provável no nó ${currentNode.data.label}`);
16    continue;
17  }
18
19  visitedStates.get(nodeId).push(stateStr);
20  arrivalHistory.get(nodeId).push({ path, state: newState });
21
22  // Enfileira sucessores lógicos satisfazendo condicionais canAccessChoice
23  // ...
24 }

```

Listagem 10: *Ciclo principal de exploração do espaço de estados no motor storyTraversal.js.*

injeta templates de prompts do sistema que constroem as respostas a respeitar a formatação estrita do Tweep 3 (Listagem 11).

```

1 // Envio do prompt de conversão estrutural para o Ollama local
2 export async function generateTweepFromText(userText, modelName = 'llama3') {
3   const response = await fetch('http://localhost:11434/api/chat', {
4     method: 'POST',
5     headers: { 'Content-Type': 'application/json' },
6     body: JSON.stringify({
7       model: modelName,
8       messages: [
9         { role: 'system', content: 'You are a parser. Convert the story to Tweep
10           3. Output ONLY raw Tweep 3 format starting with :: StoryTitle.' },
11         { role: 'user', content: userText }
12       ],
13       stream: false
14     })
15   });
16   const data = await response.json();
17   return data.message?.content || '';
18 }

```

Listagem 11: Estrutura de comunicação assíncrona com o Ollama local em *aiGenerator.js*.

1.2.3 Fluxo de Dados e Reatividade

O fluxo de dados da aplicação funciona de forma estritamente reativa e unidirecional para garantir a estabilidade do sistema e a sincronização instantânea de todos os painéis, conforme ilustrado no diagrama da Figura ??.

Quando o autor edita o texto de uma passagem no *Inspector.jsx*, um evento de alteração é propagado para o estado central em *App.jsx*. Este estado é atualizado, forçando a reavaliação de dois subsistemas independentes em segundo plano:

1. O analisador sintático extrai instantaneamente as novas arestas baseadas em ligações double-bracket e reconstrói a Lista de Adjacência Bidirecional em memória.
2. O validador estático (*storyValidator.js*) executa de imediato a travessia BFS sobre a nova lista de adjacência, atualizando o mapa de erros lógicos.

Qualquer erro detetado é propagado para o canvas do React Flow, que desenha alertas visuais de aviso nas bordas dos nós afetados. Se o utilizador abrir o *PlayMode*, o simulador acede à lista de adjacências atualizada e ao interpretador de *SugarCube* para processar as macros lógicas em tempo de execução sem requerer qualquer compilação ou carregamento manual.

1.3 Testes de Conversão e Comparação de Modelos de Linguagem (IA Local)

Para ver como o assistente de importação de texto livre se comportava, fizemos testes com diferentes modelos de inteligência artificial (LLMs) a correr localmente através do Ollama.

A primeira série de testes foi feita a **9 de maio** no computador do programador, que tem uma gráfica Nvidia RTX 3050 (Laptop) com apenas 4 GB de VRAM. O objetivo era correr localmente modelos populares com grande volume e verificar a sua viabilidade, sendo selecionados os seguintes modelos da biblioteca Ollama:

- **Gemma 2B:** ID 8ccf136fdd52
- **Mistral 7B:** ID 6577803aa9a0
- **Qwen 3.5 9B:** ID 6488c96fa5fa
- **DeepSeek Coder 6.7B:** ID ce298d984115

Além destes, também foi testado o modelo *Claude* (acessível via API externa). No entanto, devido à falta de poder de processamento da gráfica local, **nenhum dos modelos locais funcionou**, deixando o computador muito lento e dando erros de falta de memória (*out of memory*).

Para conseguir testar estes modelos, a **19 de maio**, o orientador do projeto fez os mesmos testes no seu computador pessoal, que tem melhores capacidades para correr IA. Analisámos os ficheiros de resultados e explicamos a seguir o que falhou em cada modelo para que não pudessem ser usados diretamente na nossa aplicação:

- **Mistral 7B (mistral_7b):** O resultado foi **inútil** para a aplicação. O modelo separou quase todas as frases do texto em passagens independentes (por exemplo, criando passagens separadas para “Inside” ou “To the left”). No nosso editor, isto cria dezenas de pequenos nós para uma única cena, tornando o grafo ilegível e quebrando o fluxo da história. Além disso, acrescentou barras invertidas (\) no fim de cada linha e deixou caracteres estranhos vindos do terminal (como [3D [K).
- **Gemma 4B / 2B (gemma4_e4b):** O modelo tentou criar as passagens, mas **escreveu os cabeçalhos das novas passagens (os ::)** dentro do texto de outras passagens. Como o formato Tweep 3 não suporta passagens aninhadas, o nosso parser lê isso como texto simples e não cria os nós no ecrã. Também deixou lixo do terminal (códigos ANSI).
- **Qwen 3 8B (qwen3_8b):** O resultado ficou **inutilizável devido a erros de formatação**. O modelo incluiu o seu raciocínio interno (“*Thinking...*”) mesmo ao início da resposta, o que faz com que o nosso importador falhe imediatamente ao tentar ler o ficheiro. Além disso, deixou um bloco JSON incompleto na posição das passagens (escreveu apenas 173" sem fechar), o que faz o parser de JSON falhar, e manteve alguns códigos ANSI.

- **Qwen 3.5 9B (qwen3.5_9b):** Foi o modelo que estruturou melhor a história, mas **ainda não é totalmente funcional**. Tal como o Qwen 3, escreveu o texto de pensamento (“*Thinking...*”) no início, bloqueando a importação direta. Criou também nós órfãos (como TakeLantern e DropLantern) que estão descritos no ficheiro mas não têm nenhuma ligação (seta) a apontar para eles a partir das outras cenas, deixando partes do jogo inacessíveis.

Com base nestes testes, chegámos à conclusão de que o uso de APIs externas (como a do Gemini ou do Claude) será sempre superior e mais fiável do que correr modelos locais, a não ser que o utilizador tenha uma máquina com pelo menos 8 GB de VRAM dedicados para IA. De momento, esta funcionalidade de importação por IA local ainda precisa de melhorias no futuro para ser totalmente utilizável, uma vez que ainda não encontramos o prompt ideal ou a solução perfeita para garantir que os modelos locais sigam as regras de formatação sem falhas.

1.4 Análise de Desempenho e Complexidade Algorítmica

Para validar a conformidade da aplicação com o requisito não-funcional de fluidez e desempenho (RNF03), que estipula latências de interação inferiores a 100 milissegundos, realizou-se uma avaliação da complexidade teórica e do comportamento prático do software sob diferentes cargas de dados.

1.4.1 Complexidade Algorítmica Teórica

Os módulos de cálculo do *Plot in a Pot* operam de forma síncrona no lado do cliente. A Tabela 1.2 resume a complexidade temporal e espacial assintótica (na notação Big- O) dos três algoritmos mais exigentes em termos computacionais.

Tabela 1.2: Complexidade algorítmica teórica dos subsistemas nucleares.

Subsistema / Algoritmo	Complexidade Temporal	Complexidade Espacial
Parser Bidirecional Twee 3	$O(L)$	$O(V + E)$
Validador de Espaço de Estados (BFS)	$O(V + E \cdot K)$	$O(V \cdot K + E)$
Interpolação de Arestas (Catmull-Rom)	$O(W)$	$O(W)$

Nota: L representa o número total de caracteres no ficheiro Twee; V e E indicam os vértices e arestas no grafo; W é o número de waypoints da aresta; K representa o limite regulador de estados visitados por nó ($K = 10$).

O parser opera com complexidade temporal linear $O(L)$ na leitura do texto completo. O validador BFS especial explora o espaço de estados com limite regulador K , garantindo

complexidade controlada mesmo em grafos com ciclos infinitos, uma vez que a paragem é forçada no pior cenário após K visitas redundantes ao mesmo vértice. O motor de splines Catmull-Rom opera de forma linear com base no número de waypoints da aresta (W), processando apenas as curvas ativas visíveis no ecrã.

1.4.2 Benchmarks de Desempenho Empíricos

Para medir a velocidade real de execução das rotinas da aplicação, montou-se um ambiente de testes sintéticos simulando três escalas crescentes de projetos de autoria. Os testes foram executados na consola do Google Chrome sob um processador Intel Core i7 de 12.^a geração com 16 GB de RAM. Os tempos médios obtidos após 50 execuções sucessivas encontram-se sumarizados na Tabela 1.3.

Tabela 1.3: Resultados empíricos de benchmarks de desempenho (em milissegundos).

Escala do Projeto	Parsing Twee 3	Validação Lógica (BFS)	Sincronização Canvas (React Flow)
Pequeno (10 nós / 15 arestas)	2.1 ms	4.3 ms	12.5 ms
Médio (50 nós / 80 arestas)	8.4 ms	12.8 ms	28.1 ms
Grande (150 nós / 240 arestas)	21.6 ms	34.2 ms	62.4 ms

Nota: Os valores medem o tempo de CPU decorrido desde o início da operação até ao redesenho final da interface gráfica do canvas do React Flow.

Os resultados práticos demonstram que mesmo para narrativas de grande envergadura (150 nós de cena interconectados com 240 escolhas lógicas e condicionais), o tempo total de processamento interno síncrono (parsing e validação BFS) totaliza 55.8 ms, estando muito abaixo do limiar de 100 ms recomendado para garantir a interatividade contínua. A latência extra introduzida pela renderização do React Flow (62.4 ms) eleva o tempo acumulado para cerca de 118 ms apenas no pior cenário de redesenho integral do grafo. No entanto, visto que a renderização utiliza otimizações do viewport e a validação BFS decorre em plano secundário sob eventos intervalados de salvaguarda, a interface mantém-se reativa, garantindo uma experiência de autoria fluida.

1.5 Metodologia dos Testes de Utilizador

Para validar a flexibilidade e usabilidade do *Plot in a Pot*, foi planeada e executada uma bateria de testes qualitativos e quantitativos com um grupo diversificado de participantes.

1.5.1 Perfil dos Participantes

Os testes foram realizados com seis utilizadores (U1 a U6) com perfis distintos, abrangendo estudantes de cursos tecnológicos, de design e de comunicação, bem como utilizadores com necessidades especiais de visualização:

- **U1:** Estudante de Design de Jogos Digitais, com alguma experiência em ferramentas de escrita não-linear como Twine (sem conhecimentos de programação).
- **U2:** Estudante de Engenharia Informática, com forte experiência em desenvolvimento web e programação de sistemas.
- **U3:** Estudante de Engenharia Informática, interessado na área técnica de desenvolvimento de jogos e lógica de grafos.
- **U4:** Estudante de Comunicação e Média, sem experiência prévia em ferramentas digitais de escrita não-linear.
- **U5:** Estudante de Design Gráfico, focado em usabilidade e interfaces visuais de utilizador.
- **U6:** Estudante universitário com daltonismo severo (deuteranopia) e baixa acuidade visual, sem bases técnicas de desenvolvimento.

1.5.2 Tarefas Avaliadas

Cada participante foi instruído a realizar um conjunto de quatro tarefas estruturadas na aplicação, sem qualquer ajuda externa:

1. **Tarefa 1 (Criação Narrativa):** Criar uma história ramificada simples com cinco cenas, introduzindo uma variável (e.g., chave) e uma condicional de saída baseada nessa variável.
2. **Tarefa 2 (Validação e Correção):** Importar um ficheiro `.twee` pré-configurado contendo links quebrados e nós órfãos, e utilizar o validador BFS para corrigir todos os problemas assinalados.
3. **Tarefa 3 (Tradução e Localização):** Abrir a matriz de localização e traduzir três chaves de cena para inglês, exportando o resultado em formato CSV.
4. **Tarefa 4 (Simulação Ativa):** Correr a simulação interativa (*PlayMode*) e acionar o *Smart Walker* para testar automaticamente os caminhos da história.

1.6 Resultados dos Testes e Desenvolvimento Iterativo

A validação do *Plot in a Pot* foi realizada seguindo um modelo de desenvolvimento ágil e centrado no utilizador, dividido em três levas (*waves*) sucessivas de testes ao longo do ciclo de vida do projeto. Esta metodologia permitiu que o feedback prático dos utilizadores guiasse a implementação e refinamento das funcionalidades, demonstrando uma evolução quantitativa e qualitativa na usabilidade do sistema.

1.6.1 Primeira Leva: Protótipo Inicial (Abril de 2026)

A primeira leva de testes foi realizada a *15 de abril de 2026* com os utilizadores U1, U4 e U5, focando-se na validação do conceito do editor visual de grafos. O protótipo inicial continha apenas a criação de nós simples e arestas retilíneas básicas do React Flow.

- *Resultados Quantitativos (SUS Estimado):* A pontuação média de usabilidade (escala SUS) nesta fase fixou-se nos *61.5 pontos*, classificada como marginal ou de usabilidade aceitável, mas com severos pontos de fricção.
- *Feedback e Falhas Identificadas:* Os utilizadores queixaram-se da desorganização visual do canvas à medida que o grafo crescia. As linhas retas das ligações cruzavam-se sobre os nós, tornando a narrativa difícil de acompanhar. Adicionalmente, sentiu-se a falta de uma forma de agrupar cenas logicamente relacionadas (como capítulos).
- *Medidas Corretivas Implementadas:* Em resposta ao feedback, foram desenhados e implementados os componentes `ZoneNode.jsx` (Zonas de agrupamento espacial) e o cálculo paramétrico das arestas curvas baseadas em Splines Cúbicas de Catmull-Rom com *waypoints* de arrasto manual em `graphMath.js`.

1.6.2 Segunda Leva: Protótipo Intermédio (Maio de 2026)

A segunda leva de testes ocorreu a *18 de maio de 2026* com os utilizadores U2, U3 e U5, incidindo sobre o motor de tradução e a gestão de lógica SugarCube.

- *Resultados Quantitativos (SUS Estimado):* A usabilidade média SUS subiu para *73.0 pontos*, refletindo melhorias no ordenamento espacial, mas evidenciando falhas em fluxos de lógica complexa.
- *Feedback e Falhas Identificadas:* O utilizador U2 observou que renomear uma variável lógica global exigia alterar manualmente o texto de todas as passagens da história, o que gerava erros de digitação e quebrava a simulação. Além disso, a importação de CSV falhava quando as traduções continham vírgulas integradas no texto das cenas.
- *Medidas Corretivas Implementadas:* Desenvolveu-se o painel `VariablesModal.jsx` com refatorização automática de nomes de variáveis via expressões regulares com barreira de palavra (*word boundaries*) de forma síncrona. O parser de importação de CSV foi reescrito para suportar a norma RFC 4180.

1.6.3 Terceira Leva: Protótipo Final (Junho de 2026)

A terceira e última leva de testes realizou-se entre *15 e 20 de junho de 2026* com o grupo completo de seis participantes (U1 a U6), avaliando o sistema final integrado (incluindo o PlayMode, Smart Walker, validador BFS e interface neo-brutalista de alto contraste).

Métricas de Conclusão e Tempos de Execução

Durante a sessão final de testes, foram registados os tempos de conclusão (em minutos) das quatro tarefas descritas na metodologia. Os resultados quantitativos encontram-se sumarizados na Tabela 1.4.

Tabela 1.4: Taxa de sucesso e tempo de conclusão (em minutos) das tarefas por utilizador (Leva Final).

Utilizador	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4	Taxa de Sucesso Total
U1	07:15	02:40	03:10	01:20	100%
U2	05:30	01:50	02:15	00:55	100%
U3	04:10	01:30	01:50	00:45	100%
U4	11:20	04:15	05:30	02:10	75%*
U5	09:45	03:30	04:20	01:40	100%
U6	08:30	03:10	03:40	01:15	100%
Média	07:45	02:49	03:29	01:21	95.8%

* Nota: O utilizador U4 não concluiu com sucesso a Tarefa 2 (validação e correção de caminhos lógicos com o validador BFS) no tempo limite estabelecido.

A taxa de conclusão média fixou-se em 95.8%. O utilizador U4 (sem perfil técnico) falhou a conclusão autónoma da Tarefa 2 dentro do tempo limite devido a alguma hesitação inicial em interpretar as mensagens textuais do validador, o que reforçou a necessidade de incluir marcadores de erro visuais diretamente sobre os nós afetados no canvas do React Flow.

Avaliação SUS da Leva Final

Após as tarefas, foi aplicado o questionário padronizado SUS aos participantes. As pontuações obtidas encontram-se representadas na Tabela 1.5.

Tabela 1.5: Pontuação obtida no questionário *System Usability Scale (SUS)* na leva final.

Utilizador	Pontuação SUS obtida
U1	87.5
U2	90.0
U3	92.5
U4	72.5
U5	80.0
U6	85.0
Média Global	84.6

A média global final de *84.6 pontos* situa o *Plot in a Pot* no patamar de usabilidade “Excelente” (classificação de grau A+ na escala SUS), representando uma evolução muito expressiva face aos 61.5 e 73.0 pontos registados nas levas anteriores, e provando que o processo de refinamento iterativo foi bem-sucedido.

1.6.4 Feedback Qualitativo e Usabilidade Visual

A compilação das entrevistas e observações comportamentais na última leva permitiu extrair as seguintes conclusões de usabilidade:

- **Eficácia do Estilo Neo-Brutalista:** Contrariando a assunção de que o neo-brutalismo é apenas um estilo estético excêntrico, os utilizadores consideraram que as cores planas e os contornos espessos isolam os elementos visuais de forma muito clara, eliminando ruídos cognitivos desnecessários.
- **Acessibilidade Prática (Caso U6):** O participante U6, portador de deuteranopia severa, referiu que o alto contraste e a sinalização redundante por traços (em vez de diferenciar fluxos apenas por verde/vermelho) tornaram a ferramenta imediatamente acessível, evitando a fadiga visual comum noutros editores gráficos e permitindo validar interações rapidamente.
- **Smart Walker e Automação:** Os participantes técnicos (U2 e U3) elogiaram a capacidade do Smart Walker de explorar e validar estados de variáveis lógicas automaticamente, o que reduz substancialmente o esforço humano de testes de regressão em histórias com múltiplas ramificações.

Apêndices



Manual de Utilização e Instalação

Este apêndice constitui o Manual de Utilização e Instalação oficial do *Plot in a Pot* (**Plot in a Pot (PiaP)**). Destina-se a autores de ficção interativa, designers de jogos e programadores que pretendam instalar, configurar e explorar todas as capacidades lógicas, visuais e de tradução da plataforma.

A.1 Requisitos do Sistema e Instalação Local

O **PiaP** é executado inteiramente do lado do cliente no navegador web (*Single Page Application*), mas requer uma pilha local mínima para desenvolvimento e para a execução do assistente de inteligência artificial local.

A.1.1 Instalação do Editor Web

Para executar o editor localmente, certifique-se de que tem o ambiente de execução Node.js instalado (versão 18 ou superior). Siga os seguintes passos de instalação:

1. Descomprima os ficheiros do projeto numa diretoria de trabalho.
2. Abra um terminal de comandos (cmd ou PowerShell no Windows, bash no macOS/Linux) na diretoria raiz do projeto.
3. Execute o comando de instalação de dependências:

```
npm install
```

Este comando irá descarregar e configurar todas as bibliotecas necessárias, incluindo o React 19, React Flow 11 e Tailwind CSS 3.

4. Após a conclusão bem-sucedida da instalação, inicie o servidor de desenvolvimento local através do comando:

```
npm start
```

A aplicação será inicializada e ficará disponível no seu navegador padrão no endereço `http://localhost:3000`.

A.1.2 Configuração do Servidor de IA Local (Ollama)

A funcionalidade de conversão e importação automática de histórias escritas em linguagem natural depende de um servidor local de inferência de Modelos de Linguagem de Grande Escala (LLMs). O **PiaP** suporta nativamente o *Ollama*. Para o configurar:

1. Descarregue e instale a versão oficial do Ollama a partir de <https://ollama.com/>.
2. Abra o terminal de comandos e execute o descarregamento do modelo recomendado para processamento de texto:

```
ollama pull llama3:8b
```

Pode também optar por modelos mais leves, como o `qwen2.5:7b` ou `gemma2:2b`, dependendo da memória de vídeo (VRAM) disponível no seu computador.

3. Por padrão, o Ollama executa um servidor em segundo plano que escuta na porta local `http://localhost:11434`.
4. No editor do **PiaP**, clique no botão de configurações (ícone de engrenagem) na barra superior, aceda ao painel de ligação da IA e ative a ligação local ao Ollama, especificando o modelo que descarregou.

A.2 Manual de Utilização do Editor Gráfico

A interface gráfica de grafos do **PiaP** segue uma estética neo-brutalista de alto contraste que facilita a distinção dos componentes visuais da narrativa.

A.2.1 O Canvas Interativo

O espaço central do editor representa a estrutura geral da história sob a forma de um grafo. Pode navegar no canvas utilizando as seguintes interações:

- **Deslocação (Pan):** Clique e arraste com o botão esquerdo do rato num espaço vazio do canvas.
- **Zoom:** Utilize a roda do rato para aproximar ou afastar a visualização. Pode também usar o painel do minimapa no canto inferior esquerdo para se situar no grafo.
- **Seleção de Elementos:** Clique com o botão esquerdo sobre qualquer nó de cena ou aresta para abrir as suas propriedades no painel lateral de inspeção.

A.2.2 Criação e Edição de Nós Narrativos (Cenas)

Para adicionar uma nova cena à história:

1. Clique com o botão direito do rato num espaço livre do canvas e selecione “Adicionar Nó Narrativo”, ou clique no botão correspondente na barra superior de ferramentas.
2. Um novo nó azul com contorno espesso aparecerá no canvas. Clique duas vezes no nó ou selecione-o para abrir o **Inspetor Lateral**.
3. No Inspetor, pode editar:
 - **ID da Cena:** Um identificador único em formato alfanumérico (ex: `cena_taberna`).
 - **Título da Cena:** O nome de identificação visual exibido na barra do nó no canvas (ex: `Interior da Taberna`).
 - **Conteúdo da Cena:** O texto literário que será lido pelo jogador.
 - **Etiquetas (Tags):** Palavras-chave separadas por vírgula que modificam o comportamento do nó (ex: `css` para estilos, `secreto` para ocultar da exportação).

A.2.3 Criação de Ligações e Arestas de Escolha

As arestas direcionadas do grafo representam as decisões disponíveis para o jogador no final de cada cena narrativa. Para criar uma ligação:

1. Posicione o ponteiro do rato sobre o ponto de saída circular (*handle* direito) do nó de origem.
2. Clique e arraste o cabo de ligação até ao ponto de entrada (*handle* esquerdo) do nó de destino.
3. Uma aresta curva suave será desenhada entre as cenas.
4. O texto da escolha que o jogador irá visualizar é automaticamente derivado das ligações escritas em formato double-bracket no conteúdo do nó de origem (ex: `[[Entrar na taberna|cena_taberna]]`).
5. **Waypoints Interativos:** Se pretender contornar outros nós, clique duas vezes sobre a aresta criada. Um pequeno ponto de controlo amarelo aparecerá. Arraste-o para deformar a curva de forma flexível. Pode adicionar múltiplos waypoints na mesma aresta.

A.2.4 Agrupamento em Zonas (Capítulos)

Para organizar visualmente projetos de escrita literária de grande envergadura:

1. Clique no botão “Criar Zona” na barra superior. Um retângulo cinzento com contorno pontilhado aparecerá no ecrã.
2. Atribua um nome à Zona no seu cabeçalho (ex: **Capítulo 1**).
3. Arraste nós narrativos existentes para dentro do retângulo da Zona. Estes passarão a ser filhos geométricos da zona.

4. Ao mover ou arrastar a Zona, todos os nós contidos nela deslocar-se-ão de forma solidária.
5. Pode redimensionar a Zona arrastando o indicador de escala no seu canto inferior direito.

A.3 Programação de Lógica Narrativa e Variáveis

O **PiaP** suporta a declaração de variáveis globais e condicionais lógicas nativas da macro-linguagem SugarCube.

A.3.1 Criação de Variáveis (Figma-Style Tokens)

Aceda ao menu “Variáveis” na barra superior de ferramentas:

1. Clique em “Criar Variável” e atribua um nome precedido do símbolo \$ (ex: `$tem_chave`).
2. Escolha o tipo de dados (Booleano, Numérico ou String) e defina o valor inicial por defeito.
3. A inicialização correta destas variáveis é escrita de forma automática no nó de metadados especial da história (`StoryInit`) em segundo plano.
4. **Renomeação Segura (Refatorização):** Se renomear uma variável existente no painel, o editor varre sintonizadamente todo o texto e blocos lógicos do projeto e substitui as referências antigas, garantindo que não ocorrem falhas de simulação.

A.3.2 Edição de Lógica condicional por Código

No conteúdo de qualquer nó de cena, pode inserir macros SugarCube utilizando a sintaxe de parênteses angulares duplos:

- **Modificação de Estado:** Para alterar o valor de uma variável quando o jogador visita o nó, utilize a macro `<<set>>`:

```
<<set $tem_chave = true>>
```

- **Condições de Escolhas:** Para exibir ou ocultar uma escolha com base no estado das variáveis lógicas, envolva a ligação na macro condicional `<<if>>`:

```
<<if $tem_chave>>
[[Abrir a porta trancada|cena_tesouro]]
<<else>>
A porta está trancada.
<<endif>>
```

A.3.3 Editor de Blocos Visuais

Caso não esteja familiarizado com regras de programação textual, selecione um nó e clique no botão “Editor Visual de Blocos” no Inspetor:

1. Um canvas interativo secundário aparecerá. Arraste blocos de atribuição e lógica de comparação.
2. Encaixe os blocos de forma sequencial (estilo Scratch).
3. Ao fechar o modal, as macros SugarCube correspondentes serão geradas e injetadas no local apropriado do conteúdo da cena.

A.4 Tradução e Localização Multi-idioma

A gestão de traduções no **PiaP** baseia-se numa Matriz de Tradução única que centraliza os textos dos nós.

A.4.1 Configurar Idiomas

Aceda ao menu “Matriz de Tradução”:

1. O idioma nativo padrão é exibido como a primeira coluna.
2. Clique em “Adicionar Idioma” e digite o código ISO de duas letras do novo idioma (ex: **en** para inglês, **es** para espanhol).
3. Uma nova coluna aparecerá na tabela.

A.4.2 Editar Textos na Matriz

A grelha apresenta todas as cenas narrativas nas linhas e os idiomas nas colunas:

1. Clique duas vezes em qualquer célula correspondente ao idioma secundário.
2. Digite o texto traduzido da cena e clique fora para guardar.
3. Quando a história for executada num idioma secundário, o motor lerá de forma reativa a string traduzida a partir desta matriz.

A.4.3 Importação e Exportação de Ficheiros CSV

Para enviar os ficheiros a um tradutor externo profissional:

1. Clique no botão “Exportar CSV” na Matriz de Tradução. Um ficheiro estruturado em formato tabular será descarregado.
2. O tradutor pode abrir o ficheiro no Microsoft Excel, Google Sheets ou qualquer editor de texto e preencher as colunas dos idiomas secundários.
3. Após a tradução, clique em “Importar CSV” e selecione o ficheiro atualizado. O editor atualizará automaticamente todas as células e strings na memória.

A.5 Validação Lógica e Simulação de Jogo

Antes de disponibilizar o jogo ao público, utilize os motores integrados de auditoria.

A.5.1 O Painel de Erros de Validação Estática

O validador estático analisa de forma contínua a correção topológica do grafo. Se existirem falhas lógicas:

- Os nós com erros serão destacados no canvas com uma borda pontilhada a vermelho e um sinal de alerta.
- Clique no botão “Erros de Validação” na barra superior para ver a lista estruturada de avisos:
 - **Nós Inacessíveis:** Cenas que nunca podem ser acedidas a partir do nó inicial sob nenhum caminho explorável.
 - **Ligações Partidas (Dead Ends):** Escolhas cujo ID de destino aponta para uma cena inexistente.
 - **Escolhas Bloqueadas:** Hiperligações condicionais cujas expressões lógicas nunca são satisfeitas.

A.5.2 Simulação Ativa (PlayMode) e Testes com o Smart Walker

Clique em “Testar Jogo” para abrir a sandbox de simulação:

1. Jogue a história clicando nas escolhas disponíveis no ecrã de simulação.
2. Utilize o painel de variáveis lateral para ver, em tempo real, as alterações de valores e depurar as macros lógicas.
3. **Smart Walker:** Ative o Smart Walker clicando em “Autoplay”. O simulador iniciará uma exploração automática da história. O Walker utiliza um algoritmo de procura por novidade que prioriza ramos menos visitados. Pode ver o agente a mudar de cenas e a alterar variáveis a alta velocidade no ecrã. Caso o Walker encontre um beco sem saída ou erro lógico, a simulação pausa e destaca o nó problemático.

A.6 Operações de Ficheiros e Publicação do Projeto

O editor suporta leitura e gravação no formato padrão da indústria.

A.6.1 Importar Ficheiros Twee 3

Clique em “Importar ficheiro .twee” na barra de menus e selecione um ficheiro de texto plano em formato Twee 3. O editor processará o cabeçalho e construirá automaticamente o canvas gráfico de nós e arestas correspondente.

A.6.2 Exportar e Compilar a História

Para guardar o seu trabalho de autoria ou disponibilizar o jogo final para os utilizadores:

- **Exportar Tweep 3:** Clique em “Exportar .tweep” para descarregar o ficheiro de texto estruturado. Este ficheiro preserva todos os dados posicionais das caixas e Zonas em metadados JSON compatíveis com outros compiladores de Twine.
- **Compilar para HTML:** Clique em “Compilar HTML Jogável”. O motor lógico do PiaP empacketa a história, os dicionários de traduções e a biblioteca do jogador num único ficheiro `.html`. Este ficheiro pode ser publicado em plataformas de jogos independentes (como o `itch.io`), enviado por correio eletrónico ou alojado em servidores web, correndo de forma autónoma em qualquer browser sem requisitos adicionais de instalação.

B

Guia de Sintaxe e Modelagem de Macros Lógicas

Este apêndice serve como guia de referência técnico para a especificação do formato Twee 3 e a sintaxe de macros lógicas SugarCube suportadas pelo editor do **PiaP**. Inclui exemplos práticos de modelação de mecânicas de jogo comuns em ficção interativa.

B.1 Estrutura de um Documento Twee 3

O formato Twee 3 é uma representação textual de histórias não-lineares estruturada sob a forma de passagens de texto delimitadas por cabeçalhos iniciados por duas colunas (`::`). A estrutura de um projeto compila-se em passagens de sistema obrigatórias e passagens narrativas (cenas).

B.1.1 Passagens Especiais de Sistema

Para ser considerado válido, um ficheiro Twee 3 importado no **PiaP** deve conter os seguintes nós de sistema:

- **StoryTitle:** Contém exclusivamente uma única linha de texto plano com o título geral do projeto.
- **StoryData:** Contém um bloco estruturado em formato **JavaScript Object Notation (JSON)** com as configurações globais da história. Os campos obrigatórios são o `ifid` (um identificador único de 128 bits para o projeto), o formato (`SugarCube`), e o nó de arranque (`start`):

```
1  :: StoryData
2  {
3    "ifid": "A4B7D9E0-C2B1-4F2A-8E6D-009988776655",
4    "format": "SugarCube",
5    "start": "cena_inicial",
6    "zoom": 1.0
```

```
7     }
```

- **StoryInit:** O nó lógico de inicialização. É executado uma única vez ao carregar a história na memória, antes de renderizar qualquer cena ao utilizador. Destina-se à declaração de variáveis globais e valores por defeito:

```
1     :: StoryInit
2     <<set $tem_chave = false>>
3     <<set $moedas = 0>>
4     <<set $nome_jogador = "Tomas">>
```

B.1.2 Passagens Narrativas (Cenas)

As cenas contêm o texto da história, as atribuições de variáveis em tempo de execução e as escolhas de navegação do jogador. O cabeçalho de uma passagem narrativa no **PiaP** pode incluir metadados de posicionamento posicional do canvas de grafos (X e Y) e Zonas (parentId) expressos em JSON:

```
1 :: cena_inicial
   {"position":{"x":100,"y":150},"size":{"width":250,"height":120},"parentId":"capitulo1"}
   [tag1 tag2]
2 Aqui começa a aventura. O mercado está movimentado e podes ver uma taberna.
3 [[Entrar na taberna|cena_taberna]]
4 [[Ir para o beco escuro|cena_beco]]
```

B.2 Sintaxe de Macros Lógicas (SugarCube)

O compilador JIT e interpretador integrado no editor do **PiaP** processa as seguintes macros lógicas para manipulação de variáveis e controlo de fluxo:

B.2.1 Macro de Atribuição: <<set>>

Utilizada para modificar o valor de uma variável. O **PiaP** suporta atribuições diretas, aritméticas e de strings:

- **Atribuição Booleana:** <<set \$tem_espada = true>>
- **Incrementação/Decrementação:** <<set \$vida = \$vida - 10>> ou <<set \$moedas = \$moedas + 5>>
- **Atribuição de Texto:** <<set \$armadura = "cota_de_malha">>

B.2.2 Macro de Controlo Condicional: <<if>>

Permite criar ramificações dinâmicas na história com base nas variáveis lógicas. A estrutura suporta condicionamento encadeado:

```

1 <<if $moedas >= 10>>
2   Podes comprar a poção de cura.
3   [[Comprar poção|cena_compra]]
4 <<else>>
5   Não tens moedas suficientes para falar com o mercador.
6   [[Voltar à praça|cena_praca]]
7 <<endif>>

```

B.3 Padrões Comuns de Modelação Narrativa

Apresentam-se de seguida soluções estruturadas para programar mecânicas clássicas de jogos de aventura e RPG no editor do **PiaP**:

B.3.1 Padrão 1: Inventário e Portas Trancadas

Mecânica em que o acesso a um determinado nó está bloqueado até que o jogador recolha um item de chave noutro local.

1. Inicializar a variável booleana no nó de metadados **StoryInit**:

```

1 <<set $chave_portal = false>>

```

2. Na sala onde a chave é recolhida (**cena_bau**), alterar o valor da variável:

```

1 :: cena_bau
2 Abres o baú de madeira e encontras uma chave dourada e brilhante.
3 <<set $chave_portal = true>>
4 [[Voltar ao salão|cena_salao]]

```

3. Na sala do portal (**cena_salao**), condicionar o acesso à porta trancada:

```

1 :: cena_salao
2 Estás em frente ao portal de pedra. A fechadura parece requerer uma
   chave especial.
3 [[Voltar ao bau|cena_bau]]
4 <<if $chave_portal>>
5   [[Utilizar a chave e passar o portal|cena_vitoria]]
6 <<else>>
7   O portal está firmemente trancado.
8 <<endif>>

```

B.3.2 Padrão 2: Contador de Tentativas e Fim de Jogo

Mecânica em que o jogador tem um limite de tentativas ou pontos de vida antes de ser derrotado.

1. Inicializar o contador numérico em `StoryInit`:

```
1    <<set $tentativas = 3>>
```

2. Na cena onde ocorre o erro ou armadilha (`cena_armadilha`), subtrair uma tentativa e verificar se o valor atinge zero:

```
1    :: cena_armadilha
2    O mecanismo da parede dispara flechas na tua direção! Conseguiu-te
    esquivar-te, mas perdeste energia.
3    <<set $tentativas = $tentativas - 1>>
4    <<if $tentativas > 0>>
5        Tens ainda $tentativas pontos de vida restantes.
6        [[Tentar novamente com cautela|cena_salao]]
7    <<else>>
8        As tuas forças esgotaram-se.
9        [[Game Over|cena_derrota]]
10   <<endif>>
```

B.3.3 Padrão 3: Caminhos Ocultos e Nós Secretos

Utilizado para cenas extra ou segredos da história recomendados apenas para leitores atentos.

- Para evitar que nós em desenvolvimento ou caminhos secretos estraguem a compilação ou apareçam nos relatórios de erros de caminhos órfãos, adicione a etiqueta `secreto` nas tags do nó:

```
1    :: cena_secreta {"position":{"x":500,"y":500}} [secreto]
2    Encontrei a sala secreta do programador! Parabéns!
3    [[Voltar à praça|cena_praca]]
```

O validador estático de grafos ([Pesquisa em Largura \(Breadth-First Search\) \(BFS\)](#)) do [PiaP](#) ignorará este nó ao verificar o alcance obrigatório da história, prevenindo falsos positivos de caminhos inacessíveis.

Anexos



Especificação Técnica

Normativa do Formato Twee 3

Este anexo apresenta um extrato normativo e resumo técnico da especificação aberta do formato ****Twee 3****, publicada e mantida pela *Interactive Fiction Technology Foundation* (**Interactive Fiction Technology Foundation (IFTF)**). A adoção deste formato como base de persistência do **PiaP** assegura a interoperabilidade com compiladores padrão do ecossistema Twine (como o *Tweego*).

L.1 Sintaxe Geral e Passagens

Um ficheiro Twee 3 é codificado em formato de texto plano UTF-8. A unidade estrutural básica é a **passagem**. Cada passagem inicia-se com um cabeçalho obrigatório, seguido de metadados opcionais, tags e, nas linhas seguintes, o corpo de texto da passagem.

L.1.1 Sintaxe do Cabeçalho da Passagem

O cabeçalho de uma passagem deve ser escrito numa única linha, obedecendo à seguinte gramática formal:

```
:: NomeDaPassagem [tags] {metadados}
```

- **::** – Delimitador inicial obrigatório. Deve ser posicionado no início de uma linha sem espaços precedentes.
- **NomeDaPassagem** – O título identificador único da passagem. Não pode conter os caracteres `[` ou `173` e é sensível a maiúsculas/minúsculas (*case-sensitive*).
- **[tags]** – Lista opcional de palavras-chave separadas por espaço, delimitada por parênteses retos.
- **173metadados175** – Objeto opcional em formato JSON delimitado por chavetas, contendo propriedades de layout ou de sistema.

L.2 Especificação de Metadados de Posicionamento

A especificação Twee 3 padroniza as coordenadas espaciais das passagens para permitir que editores gráficos visualizem o grafo de forma coerente.

L.2.1 Campos de Geometria JSON

Os seguintes campos são reservados dentro do objeto de metadados JSON do cabeçalho:

- **"position"** – Um objeto contendo as propriedades numéricas de posição absoluta no ecrã:
 - **"x"** – Coordenada horizontal em píxeis.
 - **"y"** – Coordenada vertical em píxeis.
- **"size"** – Um objeto opcional definindo as dimensões físicas da caixa no canvas:
 - **"width"** – Largura da caixa em píxeis.
 - **"height"** – Altura da caixa em píxeis.

No **PiaP**, para permitir o agrupamento em Zonas, estendeu-se esta especificação adicionando a propriedade **"parentId"** que armazena a string do ID da Zona mãe à qual o nó pertence geometricamente.

L.3 Especificação de Hiperligações e Transições

A especificação Twee 3 define que as arestas do grafo são declaradas de forma textual no corpo das passagens. O parser deve identificar hiperligações expressas em parênteses retos duplos (`[[...]]`). As formas válidas de declaração são:

1. **Ligação Direta:** Onde o texto exibido coincide com o ID do nó de destino:

```
[[CenaDestino]]
```

2. **Ligação com Alias (Sintaxe Clássica):** Onde se separa o texto visível do destino lógico por uma barra vertical:

```
[[Texto da Escolha|CenaDestino]]
```

3. **Ligação com Seta Direcionada:** Permite desenhar setas explícitas de navegação lógica:

```
[[Texto da Escolha->CenaDestino]]
```

O analisador léxico `tweeParser.js` do **PiaP** implementa expressões regulares robustas para extrair estes três formatos, garantindo compatibilidade bidirecional absoluta com a norma da **IFTF**.

